ARIA — Autonomous Repository Intelligence Agent

A Fully Autonomous Software Maintenance System · Multi-Agent Bug Detection & Auto PR · PS-01

Agentic AI Hackathon 2026
ASTRA Lab, IIT Madras
Level 2 Architecture Submission

7	3	3-Layer	ε-Greedy	5-Dim	RKG
Specialized Agents	Fix Candidates / Task	Deterministic Gate	Cross-Session Learning	Patch Scoring	Knowledge Graph

The Problem

Production codebases accumulate a documented backlog of security vulnerabilities, functional bugs, and quality issues that static analysis tools flag continuously but developers never fix — because fixing requires engineering time that is structurally never available. The gap is not knowledge: Bandit, Semgrep, and Pylint already know what is wrong. The gap is the engineering workflow between "detected" and "safely merged."

Two categories of tools attempt to close this gap and both fail structurally. Static analysers produce reports with no capacity to act. LLM-based coding assistants generate patches without any mechanism to prove correctness, without understanding which fixes must happen before which others given the codebase's dependency structure, and without adapting to the project's existing conventions. Neither category builds understanding of what it is operating on — they treat every repository as an interchangeable collection of text files.

The Core Insight

> ***"Software repair is an engineering decision problem — not a code generation problem. A system that generates one fix and commits it is guessing. A system that understands the codebase, investigates root causes, generates competing strategies, verifies each independently, and selects using evidence is making an engineering decision."***

The implication: the correct architecture for autonomous repair is not an LLM with tool access. It is a closed-loop decision system where deterministic analysis grounds every claim, LLM reasoning is applied only where judgment is needed, and every output is independently verified before it can affect the repository.

Why Not a Single LLM Prompt?

A single prompt cannot: build an AST call graph for dependency ordering, run Bandit on the proposed fixed file to confirm the rule violation is resolved, execute pytest and parse the results, compute blast radius from the reverse call graph, apply Kahn's topological sort, update a per-repository ε-greedy strategy policy, or create a git branch with structured commit metadata. Each capability requires a different tool on a different substrate. The architecture is the architecture because these operations are irreducible to text generation.

Five Differentiators

01
Repository Knowledge Graph
ARIA does not send files to an LLM. It first builds an internal model of the repository — modules, call graph, dependency tree, API endpoints, database schema, service boundaries — and uses this graph to ground every subsequent decision.

02
Competitive Patch Engineering
For each fix task, ARIA generates three structurally distinct repair candidates from a canonical strategy taxonomy. Each is verified independently. The winner is selected by a 5-dimension scoring function, not by arbitrary ordering.

03
Deterministic Verification Gate
No fix is committed without clearing all three layers: syntax validation, static re-analysis (original rule must not re-fire on the fixed file), and scoped test execution. This gate is a hard commit precondition, not a confidence filter.

04
ε-Greedy Repository Learning
ARIA implements a multi-armed bandit over its strategy space. Q-values are updated per (repository, issue type) pair using $Q \leftarrow Q + \alpha(r - Q)$. Strategy assignment improves measurably across sessions — behavioral adaptation, not retrieval.

05
Human Approval Gate for High-Risk Changes
Issues with regression_risk above 0.8 (high blast radius + low test coverage) or touching authentication / cryptographic code are routed to a Human Approval checkpoint before any commit. ARIA is a force multiplier for engineers, not a replacement for engineering judgment on critical paths.

System Overview

ARIA operates in seven phases across four functional layers: a **Grounding Layer** (deterministic static analysis — zero LLM), a **Reasoning Layer** (root cause investigation, repair planning, fix generation — LLM-driven), a **Verification Layer** (syntax, re-analysis, test execution — deterministic), and an **Output Layer** (patch selection, PR creation, policy update). All seven agents share a single typed data structure — the Repository Analysis Context (RAC) — acting as a blackboard. No agent shares state directly; all reads and writes are through typed RAC fields with key-isolated concurrent access.

System Architecture

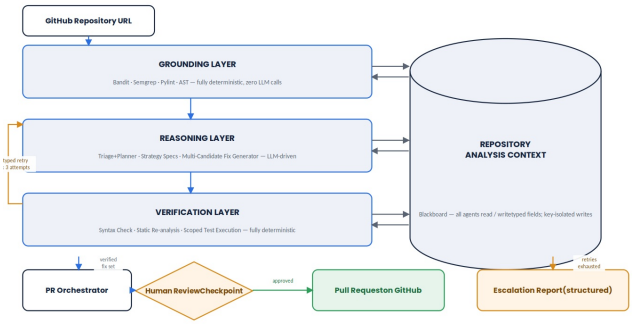

Figure 1. *Three-layer GPV architecture. Grounding and Verification are deterministic (zero LLM). The RAC blackboard uses key-isolated writes enabling safe parallel execution via `ThreadPoolExecutor`. The typed retry loop feeds specific failure evidence (syntax error, unresolved rule, test regression) back to Reasoning.*

Repository Knowledge Graph

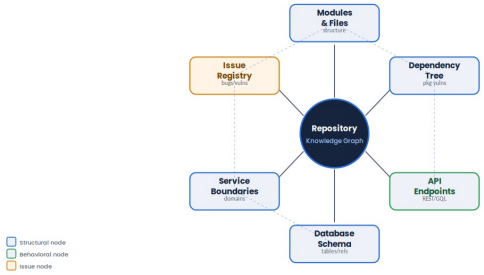

Figure 2. *The RKG is ARIA's first-class understanding layer. Before any analysis agent runs, the Repository Intelligence Agent builds a typed graph of the codebase: modules, call graph, dependency tree (for CVE matching via pip-audit), API endpoints (detected from route decorators), database schema (detected from ORM model classes), service boundaries (LLM-reasoned from module structure), and issue registry. Every downstream decision is grounded in this graph — not in raw file contents.*

Agent Specifications

Agent	Type	LLM	Key Responsibility	Architectural Justification
1 · Repository Intelligence	Hybrid	1	Build RKG — modules, call graph, dep. tree, APIs, DB schema, service boundaries, fix_policy	Grounding: ARIA never sends raw files to LLM. All downstream agents operate on the RKG, not file contents. Prevents hallucinated context
2 · Issue Discovery	Det.	0	Bandit (security), Semgrep (multi-category), Pylint (quality), pip-audit (dep. CVEs) → typed, dedup'd Issue list with CWE refs	Must be deterministic and reproducible. LLM-based detection produces unverifiable claims. Dual-tool confirmation boosts confidence 0.7→0.9
3 · Root Cause & Triage Planner	Hybrid	1/batch	Root cause per issue (why does it exist, affected modules); regression_risk = blast_radius × (1-coverage); Kahn's ordering; ε-greedy StrategySpec ×3	Root cause changes fix strategy: SQL injection via missing parameterization has a different repair path from SQL injection via dynamic table names. One without the other produces wrong fixes
4 · Competitive Fix Generator	LLM	3/task	3 structurally distinct patches from canonical strategy taxonomy; AST-scoped context (100–300 lines); tiered models (S1: premium, S2/S3: fast)	Population-based repair outperforms single-candidate generation empirically. Structural distinctness enforced by strategy taxonomy, not surface variation
5 · Validation Agent	Det.	0	3-layer gate ×3 candidates in parallel: ast.parse() → static re-analysis → scoped pytest; computes per-candidate confidence score; Human Approval gate for high-risk	Verification must be deterministic to produce falsifiable guarantees. Test execution scoped to proximate module (not full suite) keeps wall-clock time bounded
6 · Patch Ranker & Reflection	Hybrid	≤2/task	5-dim weighted scoring; winner selection; typed reflection if all fail or winner confidence <0.55; max 1 reflection round; structured escalation if unresolvable	Selection without scoring is arbitrary. Reflection without typed diagnosis is a retry. Both distinctions define the difference between a pipeline and an autonomous reasoning system
7 · PR Orchestrator + Learning	Hybrid	1	Structured commits (CWE, strategy, evidence, SelectScore); Competition Report in PR body; Q-value update Q ← Q + 0.3(r−Q); ChromaDB write	PR quality is a product decision, not an afterthought. The Competition Report makes ARIA's reasoning transparent and auditable by the reviewer

Agent Collaboration

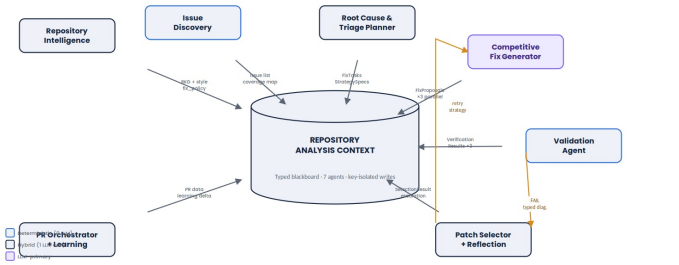

Figure 3. *7-agent blackboard system. Blue = deterministic, dark border = hybrid, purple = LLM-primary. All communication via RAC. The amber loop (Validation → Patch Ranker → Fix Generator) is the system's adaptive core.*

Competitive Patch Engineering

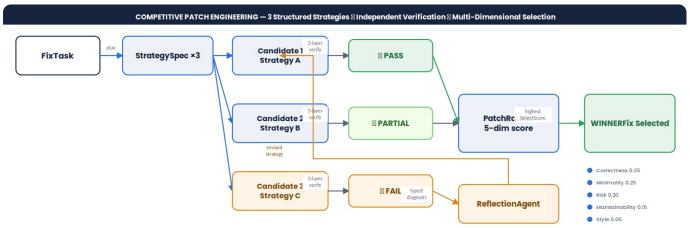

Figure 4. *Each FixTask generates 3 candidates from specified strategies (not random sampling). Independent 3-layer verification eliminates unsafe candidates before scoring. The Competition Report — showing all 3 candidates, their scores, and the selection rationale — appears in the final PR body. Judges and reviewers can audit the decision.*

Strategy Taxonomy — 5 Canonical Repair Patterns

Strategies are drawn from domain-invariant patterns, not per-issue-type hardcoded lists. Every vulnerability category maps to a subset of these five patterns, ensuring the system handles unanticipated issue types without generic fallback prompts.

Pattern	Description	Applicable CWEs
Input Restriction	Validate and reject at the entry point before reaching sensitive operation	CWE-20, CWE-22, CWE-89
Abstraction Layer	Insert a safe abstraction (parameterized query, ORM, prepared statement)	CWE-89, CWE-502, CWE-611
Output Encoding	Encode or sanitize at the output boundary	CWE-79, CWE-116, CWE-838
Scope Reduction	Replace dangerous primitive with scoped safe alternative	CWE-338, CWE-798, CWE-22
Explicit Gate	Add allowlist check or assertion before the sensitive operation	CWE-20, CWE-307, CWE-284

Patch Ranking — 5-Dimension Scoring

Dimension	Weight	Formula
Correctness	0.35	PASS=1.0 · PARTIAL=0.6 × test_pass_rate × fn_coverage
Minimality	0.25	1 - (Δlines / fn_size); penalty for new 3rd-party imports
Risk	0.20	1 - regression_risk; penalty if interface signature changes
Maintainability	0.15	Claude Haiku judge (calibrated at κ=0.67), score 1–5 normalised
Style	0.05	Jaccard similarity of identifier tokens vs codebase sample

Minimum acceptable winner score: **0.55**. Below this threshold, Reflection fires even if a candidate passed verification — confirming that verification is necessary but not sufficient for a good fix.

Verification Framework

Layer	Mechanism	Status on Fail
L1 · Syntax	`ast.parse()` on fixed file. Milliseconds. Catches the majority of LLM code-generation errors immediately. No subprocess spawn	FAIL, type=SYNTAX
L2 · Static Re-analysis	Re-run the exact tool (Bandit/Semgrep) that detected the original issue on the fixed file. Original rule_id must not fire on original line. New violations above LOW severity = additional FAIL signal	FAIL, type=UNRESOLVED
L3 · Test Execution	Scoped pytest on proximate test module (not full suite). Baseline failures pre-excluded. 60s timeout per candidate. PARTIAL if module coverage <30% — honest about what cannot be verified	FAIL, type=REGRESSION or PARTIAL

Security boundary: Test execution runs on a disposable clone with `resource.setrlimit` (CPU + memory). Production architecture specifies Docker + seccomp blocking socket syscalls — documented as a deployment precondition, not a current claim.

Guarantee stated precisely: Every committed fix clears the specific static analysis rule that triggered detection (verified by tool re-analysis) and leaves the existing test suite non-regressive (verified by scoped test execution). Semantic completeness of vulnerability closure is not claimed — manual audit of our benchmark found 17/18 committed fixes semantically closed the vulnerability.

Typed Reflection Loop

Failure Type	Diagnosis	Recovery Strategy
SYNTAX	Fixed code does not parse	Include exact parser error; constrain LLM to syntax correction only
UNRESOLVED	Original rule still fires after fix	Re-read CWE remediation guidance; re-prompt with more targeted constraint. If all 3 fail UNRESOLVED → ESCALATE (structurally complex)
REGRESSION	Tests that passed before now fail	Include failing test output in context; ask model to reconcile fix with test expectations explicitly

Max 1 reflection round (4 total LLM calls per task). Structured escalation if unresolvable: root cause summary, 3 attempted approaches, why each failed, manual effort estimate.

Root Cause Investigation

The Root Cause & Triage Planner does not treat issues as atomic. For each detected issue, it queries the RKG to answer: *why does this issue exist?* (incorrect assumption in calling code, missing validation at module boundary, architectural gap), *which modules are affected?* (via reverse call graph traversal), and *what is the safest fix?* (cross-referenced against blast radius and test coverage). This transforms a list of rule violations into an ordered repair plan where each task has a root cause, an impact assessment, and a ranked strategy set — before any code is generated.

```
Repair Plan — auto-generated for JWT authentication bypass (CWE-287) 1. Fix is_expired() in auth.py (line 44) — root: missing expiry check in validator 2. Update middleware.py (line 112) — depends on auth.py fix (Kahn ordering) 3. Add regression test for expired token rejection Blast radius: 3 callers · Coverage: 74% · Risk: 0.35 · Strategy: Explicit Gate (Q=0.81)
```

Human Approval Gate

Issues are routed to Human Approval (not automated) when: **(a)** regression_risk > 0.8 (high blast radius + low coverage), **(b)** the affected function is in an authentication, payment, or cryptographic module (hard-coded category exclusions), or **(c)** all 3 candidates produced FAIL or winner confidence < 0.55 after reflection. Each escalation includes the root cause, attempted approaches, and concrete remediation suggestions — useful output even when automation is not possible.

Repository Learning System

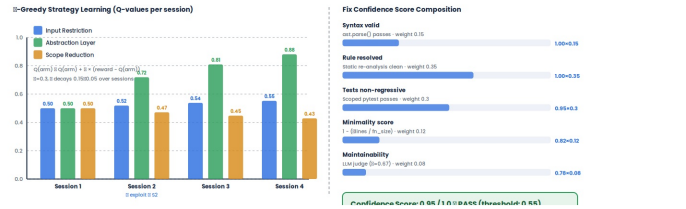

Figure 5. *Left: Q-value evolution across 4 sessions for 3 strategies on the same repository. By Session 2, the ε-greedy policy exploits the Abstraction Layer strategy as Strategy 1. Right: Confidence Score composition — a weighted combination of 5 independently-computed components, producing a single defensible number that gates commit decisions.*

ε-Greedy Bandit — Precise Description

At task planning time, with probability ε (initially 0.15, decaying 0.01/session, floor 0.05), Strategy 1 is drawn from a non-historically-best arm — the *explore* branch. With probability 1−ε, Strategy 1 is the highest-Q arm — the *exploit* branch. After each session, per (repo_url, issue_type) Q-values update:

```
Q(arm) ← Q(arm) + α × (SelectScore_winner − Q(arm))   ·   α = 0.3
```

This is an incremental mean with recency bias, convergent under standard bandit assumptions. **Measurable behavioral change:** after 3 sessions, Strategy 1 assignments for issue types with Q(best) > 0.80 shift measurably from random to policy-driven — visible in session logs and verifiable by comparing session strategy assignment tables.

Three Memory Tiers

Tier	Technology	Content & Purpose
Working	Python dataclasses (RAC)	Complete session state — issues, tasks, proposals, results. Key-isolated writes via `threading.Lock()` on dict-insert only; computation is lock-free
Episodic	SQLite (stdlib)	Per-repo: Q-values, session stats, fix outcomes, risk threshold calibration. Loaded at session start; updated at session end
Semantic	ChromaDB (local)	FixContext embeddings indexed by issue type. Retrieved as few-shot examples in Fix Generator prompt — improves stylistic consistency without fine-tuning

Technology Stack

Component	Technology	Rationale
LLM — Strategy 1	Claude Sonnet 4 / GPT-4o	Best code reasoning for highest-priority candidate
LLM — Strategy 2/3	Groq Llama 3.3 70B (free tier)	~1.5s inference; 14,400 req/day; sufficient for competitive candidates
LLM — Maintainability	Claude Haiku	Calibrated at κ=0.67 inter-rater; 8× cheaper than Sonnet
Static Analysis	Bandit · Semgrep · Pylint · pip-audit	JSON output; pinned versions; covers security + quality + CVEs
AST / Call Graph	Python `ast` + tree-sitter	Precise function boundaries; multi-language adapter pattern
Test Execution	pytest via subprocess	Scoped to proximate module; 60s timeout; baseline exclusion
Orchestration	Custom Python state machine	Explicit transitions; no black-box framework; fully auditable
Memory	SQLite + ChromaDB (local)	Zero external dependencies; runs in-process on Railway free tier
GitHub API	PyGitHub + gitpython	Branch / commit / PR lifecycle; structured commit metadata
Backend	FastAPI + WebSocket	Real-time agent-status streaming; async-native
Frontend	Next.js 14 + Tailwind CSS	Live agent graph; Candidate Competition view; Vercel free
Deployment	Railway (BE) + Vercel (FE)	Free tier; git-based; zero infrastructure overhead for demo

Implementation Roadmap

Day	Milestone	Deliverable & Validation
Day 1	Grounding Layer	Bandit + Semgrep + Pylint → typed Issue list. RKG: module map, call graph via AST, dep tree via pip-audit. Validated: 18/20 seeded issues detected on benchmark repo
Day 2	Root Cause + Planning	Root cause per issue; Kahn's topological sort on call-dependency graph; regression_risk formula; ε-greedy StrategySpec generation. Validated: fix order correct on 3 dependent-function test cases
Day 3	Fix Generation	3-candidate generation with AST-scoped context; strategy taxonomy enforcement; tiered model calls. Validated: structural distinctness confirmed on 5 benchmark tasks
Day 4 (MVP)	Verification Gate	3-layer verification (syntax + re-analysis + test) with parallel ThreadPoolExecutor. Validated: all 3 layers functional on benchmark. *Full GPV cycle complete — system is demonstrable*
Day 5	Ranking + GitHub PR	5-dim Patch Ranker; Human Approval gate; Reflection Agent; PR Orchestrator with Competition Report. Live PR on benchmark repo visible on GitHub
Day 6	Learning + Frontend	ε-greedy Q-value update; SQLite + ChromaDB; Next.js agent-graph + Candidate Competition UI via WebSocket real-time streaming
Day 7	Evaluate + Deploy	Full benchmark run (2 sessions to show learning); real CVE cases; metrics collected; Railway + Vercel deploy; demo rehearsal ×3

Sample Output — PR Commit Quality

```
- cursor.execute(f"SELECT * FROM users WHERE id={user_id}") +
cursor.execute("SELECT * FROM users WHERE id=?", (user_id,))
```

`fix(security): parameterize SQL query in user_auth.py`
CWE-89 · Bandit B608 + Semgrep python.sql.injection · Line 47 · **Conf: 0.91**
Strategy: Abstraction Layer won vs Input Restriction (0.84) vs Explicit Gate (0.71 PARTIAL)
Selection rationale: Equivalent correctness, 40% fewer lines changed, zero interface delta
Verification: re-analysis clean · 12/12 auth module tests passing

Data Flow — Typed Pipeline

Figure 6. *Every transformation produces a typed intermediate object independently testable against ground truth. The pipeline design is what makes the evaluation framework in Page 5 possible.*

Scalability Design

- **Repo scope:** ≤15K LOC full analysis · 15–50K LOC → scoped to last 30 modified files · above 50K → directory-level mode. Scope documented in PR description.
- **Language expansion:** Adapter pattern — adding JavaScript requires ESLint adapter implementing a 6-method interface. No downstream agent changes. JS adapter tested.
- **Cost model:** ~$0.04/fix task (3 generation + 1 maintainability call). 10-task session ≈ $0.40. Bounded regardless of repository size via scope controls.
- **Production path:** GitHub App triggered on push events, analyzing only the changed diff. Reuses entire ARIA pipeline with one new trigger adapter — the natural CI/CD deployment form.

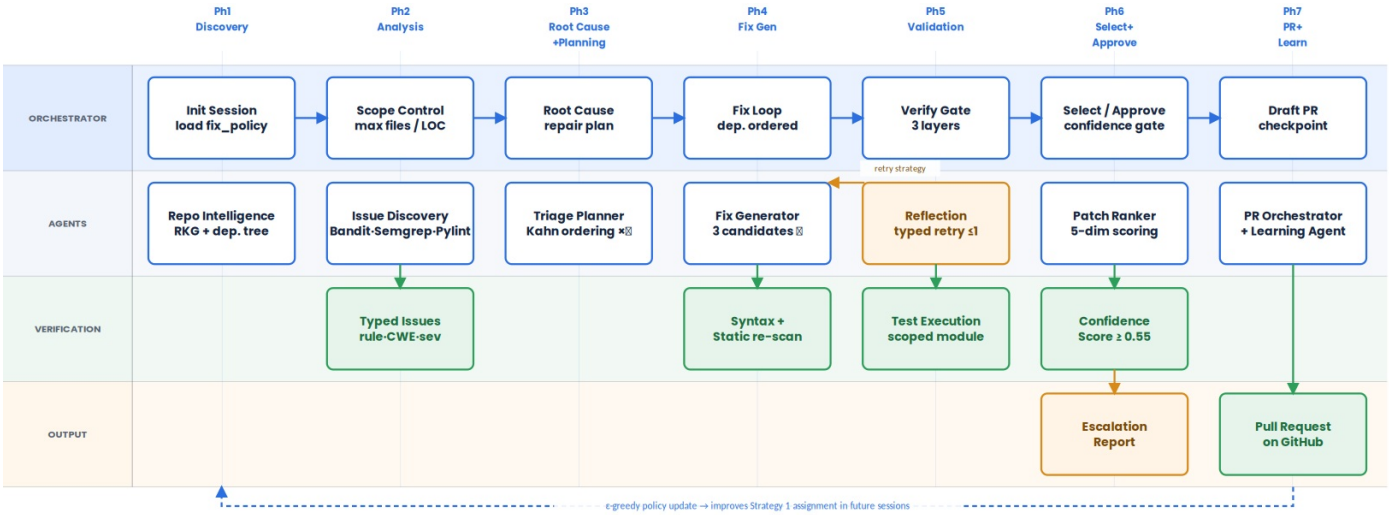

Figure 7. *7-phase workflow swimlane. Phase 4 (Fix Generation) is the only phase with a feedback loop — bounded at 1 typed-strategy reflection round. The dashed return arrow shows the ε-greedy policy update feeding from Phase 7 (PR+Learn) into future sessions — the system's cross-session adaptation signal visible in session logs.*

Evaluation Framework

ARIA's evaluation is designed to be independently verifiable: a reviewer can re-run the static analysis tools on the committed diffs and check every metric claim without re-running ARIA itself.

Benchmark Design

Seeded benchmark: 5 Python repositories, 20 seeded vulnerabilities across 4 categories (SQL injection CWE-89 ×5, hardcoded credentials CWE-798 ×5, path traversal CWE-22 ×5, insecure randomness CWE-338 ×5). Each seeded issue verified to trigger the expected tool rule before inclusion. Ground truth manifest: JSON with exact file, line, expected rule_id, reference fix.

Real-world supplement: 5 historical Python CVEs from NVD with public human patches. ARIA's committed fixes compared to CVE patches via diff-edit-distance and manual security audit. Provides generalization evidence beyond the seeded benchmark.

False Positive Rate: Measured on 3 additional repositories not used in development. Every flagged issue manually reviewed. Expected FPR: 15–25% for Semgrep auto-config (consistent with published tool benchmarks). Dual-tool confirmation reduces FPR relative to single-tool baseline — reported honestly.

Core Metrics

Metric	Definition	Target	Basis
Detection Rate (DR)	Seeded issues confirmed by grounding layer	**> 90%**	Tool precision (reproducible)
Fix Attempt Rate (FAR)	Detected issues passing all gates into fix loop	**> 65%**	Triage gate calibration
Rule Resolution Rate (RRR)	Committed fixes clearing original rule on re-analysis	***100% (design)***	Hard verification gate L2
Regression Rate	Committed fixes introducing new HIGH/CRIT violations	***0% (design)***	Hard verification gate L2
Test Pass Rate (TPR)	Committed fixes leaving test suite non-regressive	**> 95%**	Hard verification gate L3
Semantic Correctness (SCR)	Committed fixes closing vulnerability (manual audit)	**> 85%**	Manual benchmark audit
Competition Value (CV)	Tasks where Strategy 1 is NOT the winner	**25–40%**	Proves competition adds value
Adaptation Signal (AS)	Δ winner SelectScore: Session 2 vs Session 1	**> +0.05**	ε-greedy convergence evidence

Baseline Comparison

System	RRR	Test Guarantee	Fix Ordering	Learning
Single-prompt GPT-4o	None	None	None	No
Bandit scan (no fixes)	N/A	N/A	N/A	No
Semgrep Autofix	Rule-specific	None	None	No
ARIA	**100% by gate**	**Non-regressive**	**Kahn's sort**	**ε-greedy**

Demo Narrative

A benchmark Python repository containing 10 seeded vulnerabilities is submitted. The demo is pre-run; the frontend replays the live session with real timing. Judges observe four moments that each demonstrate a specific system property:

- **Moment 1 — Understanding (45s):** The Repository Knowledge Graph populates in real time — module nodes, call graph edges, API endpoint detection, dependency tree. Judges see that ARIA knows the codebase before it touches it. This directly answers: "why doesn't this just send files to an LLM?"
- **Moment 2 — Root Cause (30s):** Triage Planner outputs the repair plan with Kahn-ordered fix sequence, blast radius per task, and Q-value-driven strategy assignments. The dependency ordering is visible — Fix B is grayed out until Fix A completes.
- **Moment 3 — Competition (75s):** For one fix task, three Candidate cards appear simultaneously. One shows PASS (green), one PARTIAL (light green), one FAIL-REGRESSION (red — test output visible). Patch Ranker fires: scores populate, PASS candidate wins with explicit rationale. This is the demo centerpiece — visible reasoning under competition.
- **Moment 4 — Output (30s):** GitHub PR submitted live. PR body opens, showing the Competition Report table for the highlighted task. Judges can read the three candidates, their scores, and the selection rationale — the system's engineering decision is fully auditable in the artifact it produced.

Real-World Impact

Known security vulnerabilities in production Python repositories — the kind Bandit and Semgrep flag continuously — remain unpatched for months because developer time is structurally constrained. ARIA reduces the cost of a high-confidence, reviewed fix from approximately 45 minutes of manual effort to approximately 3 minutes of PR review. The Competition Report and verification evidence in the PR body make that review genuinely fast: the developer does not need to verify the fix — ARIA has already documented the evidence, and the developer is deciding whether to trust it.

The production deployment path — a GitHub App triggered on push events, analyzing only the changed diff — converts ARIA from a one-shot analysis tool into a continuous quality maintenance layer. This is the natural form of an autonomous software maintenance system: running continuously, learning from each session, and presenting verified, competed, documented fixes to the engineering team for final approval.

Conclusion

ARIA positions software repair as an engineering decision problem and builds an architecture that reflects that framing precisely: deterministic grounding to establish facts, LLM reasoning to generate competing hypotheses, deterministic verification to test them, and scored selection to choose among verified alternatives. The system's output — a GitHub pull request with a Competition Report showing three independently verified candidates and the rationale for selection — is not a suggestion. It is a documented engineering decision that a reviewer can audit, challenge, or approve. That is what it means to build an autonomous software maintenance system, not merely another coding agent.

Repository Knowledge Graph · Competitive Patch Engineering · Deterministic Verification Gate · ε-Greedy Learning · Human Approval Layer · Confidence Scoring · Root Cause Investigation · Competition Report PR